Assignment No. 8

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git

---

Optimizing Delivery Routes for a Logistics Company Using Least Cost Branch and Bound

1. Aim

To implement the Travelling Salesman Problem (TSP) using the Least Cost (LC) Branch and Bound technique to determine the minimum-cost delivery route that visits every city exactly once and returns to the starting city.

2. Problem Statement

SwiftShip, a logistics company, needs to optimize its delivery routes to reduce transportation costs. A delivery vehicle must visit every city exactly once and return to its starting location. Each route between two cities has an associated travel cost.

The objective is to find the minimum-cost tour using the Least Cost Branch and Bound algorithm. The algorithm uses a reduced cost matrix and a priority queue to explore promising partial routes while pruning branches that cannot improve the best known solution.

3. Course Objectives

1. To understand the computational complexity of different algorithms.
2. To select appropriate algorithm design strategies for solving real-world problems.

4. Course Outcomes

1. Apply the Branch and Bound technique to solve optimization problems.
2. Classify problems into P, NP, NP-Complete, and NP-Hard categories.

5. Theory

5.1 Travelling Salesman Problem

The Travelling Salesman Problem is a combinatorial optimization problem in which a salesperson or delivery vehicle must visit every city exactly once and return to the starting city while minimizing the total travel cost.

For (N) cities, a brute-force approach may examine ((N-1)!) possible tours when the starting city is fixed. This becomes computationally expensive as the number of cities increases.

5.2 Branch and Bound

Branch and Bound is an algorithm design technique used to solve optimization problems by systematically exploring candidate solutions.

It consists of three main operations:

- **Branching:** Divide the problem into smaller subproblems by selecting the next city.
- **Bounding:** Calculate a lower bound on the minimum cost achievable from a partial route.
- **Pruning:** Discard a branch when its lower bound cannot improve the best complete solution found so far.

5.3 Least Cost Branch and Bound

In Least Cost Branch and Bound, each search-tree node represents a partial route. Nodes are stored in a priority queue, ordered by their lower-bound costs.

The node with the smallest bound is expanded first. This best-first strategy prioritizes routes that appear more promising and can reduce unnecessary exploration.

Each node maintains:

- The reduced cost matrix.
- The lower-bound cost.
- The current city and search level.
- The partial route.
- The actual cost accumulated along the partial route.

5.4 Cost Matrix Reduction

A cost matrix represents the travel cost between every pair of cities. The diagonal entries are treated as infinity because travelling from a city to itself is not allowed as a normal transition.

Matrix reduction involves:

1. **Row reduction:** Subtract the smallest finite value in each row from every finite value in that row.
2. **Column reduction:** Subtract the smallest finite value in each column from every finite value in that column.
3. **Lower-bound calculation:** Add the values subtracted during row and column reduction.

The reduction value contributes to a lower bound on the cost of a complete tour. When a route edge is selected, the algorithm updates the matrix to prevent invalid transitions and calculates the child's new bound.

Important: The lower bound is used to guide the search and prune branches; it is not necessarily the actual cost of the completed tour. The final tour cost is calculated using the original edge costs.

5.5 Pruning

Let the lower bound of a node be (LB), and let the best complete tour cost found so far be (C_{best}).

If

[
LB \geq C_{best}
]

the node can be pruned because it cannot produce a strictly better solution.

5.6 Complexity

The Travelling Salesman Problem is NP-Hard. Branch and Bound can significantly reduce the search space, but it does not eliminate the factorial worst-case complexity.

6. Algorithm: LC Branch and Bound

1. Initialize the cost matrix, representing unavailable self-transitions with infinity.
2. Reduce the initial matrix and calculate the root's lower bound.
3. Create a root node containing the starting city and insert it into a minimum-priority queue.
4. Remove the node with the smallest lower bound.
5. If the node cannot improve the best known solution, prune it.
6. Otherwise, generate child nodes for unvisited cities.
7. For each child, update the reduced matrix, calculate the new lower bound, and update the accumulated actual route cost.
8. Insert promising children into the priority queue.
9. When a route includes all cities, add the cost of returning to the starting city and compare its total cost with the best known tour.
10. Continue until the priority queue is empty.
11. Display the optimal tour and its total cost.

7. Pseudocode

```
LC_TSP_BranchAndBound(costMatrix, start):

    Reduce the initial cost matrix
```

```
    Create root node with its lower bound
    Insert root into a minimum-priority queue

    bestCost = INFINITY
    bestPath = NULL

    while priority queue is not empty:

        node = remove node with minimum lower bound

        if node.lowerBound >= bestCost:
            continue

        if all cities have been visited:
            totalCost = node.pathCost
                        + cost(lastCity, start)

            if totalCost < bestCost:
                bestCost = totalCost
                bestPath = node.path + start

            continue

        for each unvisited city:

            Create a child node
            Update its reduced cost matrix
            Calculate its lower bound
            Update its actual path cost

            if child.lowerBound < bestCost:
                insert child into priority queue

    return bestPath, bestCost
```

8. Sample Input

The program uses four cities: A, B, C, and D. The following symmetric cost matrix represents the travel costs between them.

From / To	A	B	C	D

A	INF	10	15	20
B	10	INF	35	25
C	15	35	INF	30
D	20	25	30	INF

Here, INF indicates that a city cannot transition to itself.

9. Expected Output

The program determines the following optimal delivery tour:

A → B → D → C → A

The total cost is:

[
10 + 25 + 30 + 15 = 80
]

Therefore, the minimum tour cost is **80 units**.

The program also displays the number of expanded and pruned nodes to illustrate how Branch and Bound reduces the search effort.

10. Complexity Analysis

Let (N) represent the number of cities.

- **Best case:** Effective pruning may allow the algorithm to explore relatively few nodes.
- **Worst-case time complexity:** (O(N! \times N^2)) for a straightforward implementation that performs (O(N^2)) matrix reduction at each search node.
- **Space complexity:** Up to (O(N! \times N^2)) in the worst case when the priority queue retains many nodes, each storing an (N \times N) matrix.

Actual performance depends on the cost matrix and the effectiveness of the lower-bound calculations.

11. Applications

1. Delivery route optimization in logistics companies.
2. Planning routes for sales representatives.
3. Manufacturing and production scheduling.
4. Circuit-board drilling and other sequencing problems.
5. Transportation and distribution planning.

12. Conclusion

The Least Cost Branch and Bound technique was implemented to solve the Travelling Salesman Problem. The algorithm uses a reduced cost matrix, lower-bound calculations, and a minimum-priority queue to explore promising routes and prune branches that cannot improve the current best solution.

For the four-city example, the program finds the optimal tour **A → B → D → C → A**, with a minimum total cost of **80 units**. Although Branch and Bound can substantially reduce unnecessary computations, its worst-case complexity remains factorial. It is therefore most suitable for exact solutions when the number of cities is moderate.
